Final Exam Preparation: Level 0 - 2 Solutions

Reference: LEVEL 0-2_2.pdf

Below are the correct C solutions for all exercises in Level 0 through Level 2, as described in your document.

Level 0 & 1 (String Traversal & Basics)

aff_a

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            if (argv[1][i] == 'a')
            {
                write(1, "a", 1);
                break;
            }
            i++;
        }
        write(1, "\n", 1);
    }
    else
    {
        write(1, "a\n", 2);
    }
    return (0);
}
```

aff_z

```
#include <unistd.h>

int main(int argc, char **argv)
{
    (void)argc;
    (void)argv;
    write(1, "z\n", 2);
    return (0);
}
```

ft_countdown

```
#include <unistd.h>

int main(void)
{
    write(1, "9876543210\n", 11);
    return (0);
}
```

hello

```
#include <unistd.h>

int main(void)
{
    write(1, "Hello World!\n", 13);
    return (0);
}
```

maff_alpha

```
#include <unistd.h>

int main(void)
{
```

```
    write(1, "aBcDeFgHiJkLmNoPqRsTuVwXyZ\n", 27);
    return (0);
}
```

only_z

```
#include <unistd.h>

int main(void)
{
    write(1, "z", 1);
    return (0);
}
```

first_word

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i] == ' ' || argv[1][i] == '\t')
            i++;
        while (argv[1][i] && argv[1][i] != ' ' && argv[1][i] != '\t')
        {
            write(1, &argv[1][i], 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

ft_putstr

```
#include <unistd.h>

void ft_putstr(char *str)
{
    int i = 0;
    while (str[i])
    {
        write(1, &str[i], 1);
        i++;
    }
}
```

ft_strcpy

```
char *ft_strcpy(char *s1, char *s2)
{
    int i = 0;
    while (s2[i])
    {
        s1[i] = s2[i];
        i++;
    }
    s1[i] = '\0';
    return (s1);
}
```

ft_strlen

```
int ft_strlen(char *str)
{
    int i = 0;
    while (str[i])
        i++;
    return (i);
}
```

ft_swap

```
void ft_swap(int *a, int *b)
{
    int tmp = *a;
    *a = *b;
    *b = tmp;
}
```

repeat_alpha

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            int repeat = 1;
            if (argv[1][i] >= 'a' && argv[1][i] <= 'z')
                repeat = argv[1][i] - 'a' + 1;
            else if (argv[1][i] >= 'A' && argv[1][i] <= 'Z')
                repeat = argv[1][i] - 'A' + 1;

            while (repeat--)
                write(1, &argv[1][i], 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

rev_print

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int len = 0;
        while (argv[1][len])
            len++;
        while (--len >= 0)
            write(1, &argv[1][len], 1);
    }
    write(1, "\n", 1);
    return (0);
}
```

rot_13

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            char c = argv[1][i];
            if ((c >= 'a' && c <= 'm') || (c >= 'A' && c <= 'M'))
                c += 13;
            else if ((c >= 'n' && c <= 'z') || (c >= 'N' && c <= 'Z'))
                c -= 13;
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
```

```
    return (0);
}
```

search_and_replace

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 4 && !argv[2][1] && !argv[3][1])
    {
        int i = 0;
        while (argv[1][i])
        {
            if (argv[1][i] == argv[2][0])
                write(1, &argv[3][0], 1);
            else
                write(1, &argv[1][i], 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

ulstr

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            char c = argv[1][i];
            if (c >= 'a' && c <= 'z')
                c -= 32;
```

```
            else if (c >= 'A' && c <= 'Z')
                c += 32;
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

Level 2 (Advanced Strings, Bitwise, & Logic)

alpha_mirror

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            char c = argv[1][i];
            if (c >= 'a' && c <= 'z')
                c = 'z' - c + 'a';
            else if (c >= 'A' && c <= 'Z')
                c = 'Z' - c + 'A';
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

camel_to_snake

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            char c = argv[1][i];
            if (c >= 'A' && c <= 'Z')
            {
                write(1, "_", 1);
                c += 32;
            }
            write(1, &c, 1);
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

do_op

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char **argv)
{
    if (argc == 4)
    {
        int a = atoi(argv[1]);
        int b = atoi(argv[3]);
        char op = argv[2][0];

        if (op == '+')
```

```
            printf("%d", a + b);
        else if (op == '-')
            printf("%d", a - b);
        else if (op == '*')
            printf("%d", a * b);
        else if (op == '/')
            printf("%d", a / b);
        else if (op == '%')
            printf("%d", a % b);
    }
    printf("\n");
    return (0);
}
```

ft_atoi

```
int ft_atoi(const char *str)
{
    int i = 0;
    int sign = 1;
    int res = 0;

    while (str[i] == ' ' || (str[i] >= 9 && str[i] <= 13))
        i++;

    if (str[i] == '-' || str[i] == '+')
    {
        if (str[i] == '-')
            sign = -1;
        i++;
    }

    while (str[i] >= '0' && str[i] <= '9')
    {
        res = res * 10 + (str[i] - '0');
        i++;
    }
```

```
    return (res * sign);
}
```

ft_strcmp

```
int ft_strcmp(char *s1, char *s2)
{
    int i = 0;
    while (s1[i] && s1[i] == s2[i])
        i++;
    return ((unsigned char)s1[i] - (unsigned char)s2[i]);
}
```

ft_strdup

```
#include <stdlib.h>

char *ft_strdup(char *src)
{
    int len = 0;
    char *dup;

    while (src[len])
        len++;

    dup = (char *)malloc(sizeof(char) * (len + 1));
    if (!dup)
        return (NULL);

    int i = 0;
    while (src[i])
    {
        dup[i] = src[i];
        i++;
    }
    dup[i] = '\0';
```

```
    return (dup);
}
```

ft_strrev

```
char *ft_strrev(char *str)
{
    int i = 0;
    int len = 0;
    char tmp;

    while (str[len])
        len++;

    while (i < len / 2)
    {
        tmp = str[i];
        str[i] = str[len - 1 - i];
        str[len - 1 - i] = tmp;
        i++;
    }
    return (str);
}
```

inter

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 3)
    {
        int seen[256] = {0};
        int i = 0;

        while (argv[2][i])
        {
            seen[(unsigned char)argv[2][i]] = 1;
            i++;
```

```
        }

        i = 0;
        while (argv[1][i])
        {
            if (seen[(unsigned char)argv[1][i]] == 1)
            {
                write(1, &argv[1][i], 1);
                seen[(unsigned char)argv[1][i]] = 2; // Prevent
duplicates
            }
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

is_power_of_2

```
int is_power_of_2(unsigned int n)
{
    if (n == 0)
        return (0);
    return ((n & (n - 1)) == 0);
}
```

last_word

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        int start = -1;
        int end = -1;
```

```
        while (argv[1][i])
        {
            if (argv[1][i] != ' ' && argv[1][i] != '\t')
            {
                start = i;
                while (argv[1][i] && argv[1][i] != ' ' && argv[1][i]
!= '\t')
                    i++;
                end = i;
            }
            else
            {
                i++;
            }
        }

        if (start != -1)
        {
            while (start < end)
            {
                write(1, &argv[1][start], 1);
                start++;
            }
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

max

```
int max(int *tab, unsigned int len)
{
    if (len == 0)
        return (0);

    unsigned int i = 1;
    int max_val = tab[0];
```

```
    while (i < len)
    {
        if (tab[i] > max_val)
            max_val = tab[i];
        i++;
    }
    return (max_val);
}
```

print_bits

```
#include <unistd.h>

void print_bits(unsigned char octet)
{
    int i = 8;
    unsigned char bit;

    while (i--)
    {
        bit = ((octet >> i) & 1) + '0';
        write(1, &bit, 1);
    }
}
```

reverse_bits

```
unsigned char reverse_bits(unsigned char octet)
{
    unsigned char res = 0;
    int i = 8;

    while (i--)
    {
        res = (res << 1) | (octet & 1);
        octet >>= 1;
    }
```

```
    return (res);
}
```

snake_to_camel

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 2)
    {
        int i = 0;
        while (argv[1][i])
        {
            if (argv[1][i] == '_')
            {
                i++;
                if (!argv[1][i])
                    break;
                char c = argv[1][i];
                if (c >= 'a' && c <= 'z')
                    c -= 32;
                write(1, &c, 1);
            }
            else
            {
                write(1, &argv[1][i], 1);
            }
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

swap_bits

```
unsigned char swap_bits(unsigned char octet)
{
```

```
    return ((octet >> 4) | (octet << 4));
}
```

union

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 3)
    {
        int seen[256] = {0};
        int i = 0;

        while (argv[1][i])
        {
            if (!seen[(unsigned char)argv[1][i]])
            {
                write(1, &argv[1][i], 1);
                seen[(unsigned char)argv[1][i]] = 1;
            }
            i++;
        }

        i = 0;
        while (argv[2][i])
        {
            if (!seen[(unsigned char)argv[2][i]])
            {
                write(1, &argv[2][i], 1);
                seen[(unsigned char)argv[2][i]] = 1;
            }
            i++;
        }
    }
    write(1, "\n", 1);
    return (0);
}
```

wdmatch

```
#include <unistd.h>

int main(int argc, char **argv)
{
    if (argc == 3)
    {
        int i = 0;
        int j = 0;

        while (argv[2][j] && argv[1][i])
        {
            if (argv[2][j] == argv[1][i])
                i++;
            j++;
        }

        if (!argv[1][i])
        {
            i = 0;
            while (argv[1][i])
            {
                write(1, &argv[1][i], 1);
                i++;
            }
        }
    }
    write(1, "\n", 1);
    return (0);
}
```